Efficient ResNets for CIFAR-10 Under 5M Parameters: What Helped and What the Private Leaderboard Taught Us

Viral Dalal Aditya Patil Shashwat Shah
Team *Gulab jamun* – NYU Tandon Deep Learning, Fall 2026

Code: https://github.com/Viral0401/DL_Project_1

Abstract

We trained residual networks from scratch to classify a held-out, CIFAR-10-style test set under a hard limit of 5 million parameters. Our final model is a 4.71M-parameter ResNet (three stages, 4/4/3 basic blocks, 64/128/256 channels) trained with self-supervised rotation label augmentation (SLA+SD), strong data augmentation (TrivialAugment and Cutout), label smoothing and a full warm-up + cosine learning-rate schedule. Relative to our team baseline, this recipe raised our best hidden-test accuracy from 85.3% to 91.5% on the public leaderboard and from 81.2% to 87.7% on the private leaderboard, a gain of about 6.5 points that held up on unseen data (our two selected submissions scored 86.5% and 86.7% privately). On top of this recipe we tested knowledge distillation, a larger number of teachers, Sharpness-Aware Minimization and Stochastic Weight Averaging. They moved the public score by up to ± 1 point but none of them produced a measurable private-leaderboard gain. The most useful outcomes of the project are therefore a set of lessons: the low-learning-rate end of a schedule is where most accuracy is earned; validation accuracy overstates accuracy on a shifted test set; a 1,000-image public leaderboard cannot distinguish models that differ by less than about 1 point, and choosing final submissions by public score invites a "winner's curse".

1 Overview

The task was to design a ResNet [1] with at most 5M trainable parameters and train it from scratch on the 50,000 CIFAR-10 training images (no pretrained weights, no external data). Models were scored by accuracy on 2,000 unlabeled test images, half used for a public leaderboard during the competition and half for the final private ranking.

Two properties of this setting shaped everything we did:

- **The test set is shifted.** Every model we trained scored 7–10 points lower on the hidden test set than on a held-out 5k split of the training data (e.g. 97.9% validation vs. 90.9% public). This is consistent with a newly collected CIFAR-10-like test set, where accuracy drops are well documented [2]. Accuracy on such sets still rises with validation accuracy, but with a lower intercept, so ideas that improve generalization rather than fitting matter most.
- **The leaderboard is noisy.** With roughly 1,000 images per half, one image is 0.1% and the standard error of an accuracy near 90% is $\sqrt{0.9 \cdot 0.1/1000} \approx 0.95$ points. Differences below about 1 point between two submissions are mostly chance.

Summary of findings. (i) Completing the learning-rate schedule and using rotation self-supervision with strong augmentation were the two largest real improvements. (ii) Seven runs of that recipe (Tries 2–8) landed between 86.5% and 87.7% on the private set: the recipe had converged and later tweaks only reshuffled noise. (iii) Our final selected submissions were the two best on the public leaderboard (91.5%, 91.4%) and scored 86.5% and 86.7% privately, near the bottom of that range. Section 5 explains why this is expected.

2 Final Model

Our network is a CIFAR-style ResNet (Table 1). Each residual block computes $\mathrm{ReLU}(S(x) + F(x))$ with F = conv3×3–BN–ReLU–conv3×3–BN, and S the identity or, when the shape changes, a 1×1 projection with BN. A small second linear head with 40 outputs (10 classes × 4 rotations) is used by the self-supervised objective (Section 3.2). It is part of the submitted model and is counted in the parameter total.

Stage	Output size	Channels	Blocks	Parameters
Stem (3×3 conv, BN, ReLU)	32×32	64	–	1,856
Stage 1	32×32	64	4	295,936
Stage 2 (stride 2)	16×16	128	4	1,116,416
Stage 3 (stride 2)	8×8	256	3	3,280,384
Global average pool + class head	1×1	10	–	2,570
Rotation (joint) head	1×1	40	–	10,280
Total (all trainable)				**4,707,442**

Table 1: Final architecture (all submitted models from Try 2 onward share it). 70% of the budget sits in the last stage, where features are richest and spatial size is smallest.

Why this shape. Starting from the team baseline (3/3/3 blocks, 32-channel stem, 4.31M parameters), we added one block to each of the first two stages and widened the stem to 64 channels, which removes a projection shortcut in Stage 1. This uses most of the remaining budget in the cheap, high-resolution stages; widening Stage 3 further would have exceeded 5M parameters. We deliberately kept the post-activation block from the assignment description; pre-activation blocks mainly help much deeper networks than our 24-layer model.

3 Methodology: The Ideas That Mattered

3.1 Finish the learning-rate schedule

All runs use Nesterov SGD (momentum 0.9), a 5-epoch linear warm-up to a peak learning rate of 0.2 (batch 256, linear scaling from 0.1 at batch 128), then cosine decay [7] to 10^{-4} over 200 epochs, with weight decay $5 \cdot 10^{-4}$ on convolution/linear weights only [8].

Our first "improved" run (larger model, RandAugment, MixUp) scored *lower* than the baseline (85.2% vs. 85.3% public). The cause was not the ideas but the schedule: the session ended at epoch 161 of 200, while the learning rate was still about 0.01. Figure 1 shows why this is costly: validation accuracy rose from about 95% to 97.9% only in the last ∼30% of the schedule, as the learning rate

fell from 0.05 towards zero. A 60-epoch schedule that finishes beats a 200-epoch schedule stopped four-fifths of the way through. After this, every run printed a projected training time after the first epoch, saved a checkpoint every epoch and could resume exactly.

3.2 Self-supervised label augmentation (SLA+SD)

Our largest single gain came from SLA+SD [3]. Every training image is shown at 0°, 90°, 180° and 270°, and the joint head predicts the 40-way label (class, rotation). The normal 10-way head sees only the upright image and is trained with cross-entropy plus a self-distillation term that matches the rotation-averaged prediction of the joint head. The intuition is that recognizing *which way up* a cat is requires knowing what a cat's shape looks like, so the shared features rely more on object structure and less on texture or color statistics, which is exactly what a shifted test set perturbs. *Cost:* four forward passes per image, so each epoch is about 4× more expensive. *Parameters:* only the 10,280-parameter joint head.

3.3 Strong but simple augmentation, label smoothing, more data, EMA

- **TrivialAugment** [4] (one random operation of random strength per image) plus random crop with reflection padding, horizontal flip and a 12-pixel **Cutout** [5]. These replace the RandAugment + MixUp combination from our first attempt; TrivialAugment needs no tuning, and mixing labels interacts poorly with the 40-way rotation labels.
- **Label smoothing** 0.1 [6] on the class head (our first attempt had removed it).
- **Training on all 50k images** once the recipe was fixed, keeping the 45k/5k split only for measurement.
- An **exponential moving average** of the weights (decay 0.999) [9]. Its predictions agreed with the raw weights on 99.4% of test images, so it was harmless but not a source of accuracy.
- **Test-time augmentation** by averaging the original and mirrored image: +0.14 points on validation (7 of 5,000 images); 1-pixel shifts and the rotation-aggregate prediction added nothing further.

3.4 Engineering that made the experiments possible

The first improved run spent 141 s per epoch on a T4, mostly waiting for PNG decoding. Loading all images into memory once, using four data-loader workers, enabling cuDNN autotuning, mixed precision and both GPUs on Kaggle brought SLA training (4× more compute) to about 100 s per epoch. Later runs on an A100 took 15–25 s per epoch. Long runs were repeatedly killed by the platform after 3.5–4.5 hours without any Python error; per-epoch checkpoints with exact resumption (model, optimizer, schedule, EMA, random state) meant these interruptions cost minutes rather than whole runs.

4 Experiments and Results

Table 2 lists every model we submitted. Validation accuracy is on the same fixed 5k split of the training data (only available for runs that held it out); public and private columns are the competition scores.

Run	Change relative to the previous step	Params	Val.	Public	Private
Baseline	ResNet 3/3/3, crop+flip+erasing, 60 epochs, 45k	4.31M	94.88	85.3	81.2
Improved v1	4/4/3, RandAugment, MixUp; *stopped at epoch 161/200*	4.68M	94.42	85.2	81.7
Try 2	SLA+SD, TrivialAugment, Cutout, label smoothing, full 200 ep.	4.71M	97.88[a]	90.9	87.1
Try 3	+ EMA, all 50k images	4.71M	–	90.2	**87.7**
Try 4⋆	+ distillation from Try 2 and Try 3 (2 teachers)	4.71M	–	**91.5**	86.5
Try 5	distillation from 3 teachers (Tries 2–4), 180 ep.	4.71M	–	91.0	87.1
Try 6	distillation from 4 teachers, mirrored teacher targets	4.71M	–	89.6	86.8
Try 7	Try 4 recipe + SAM ($\rho = 0.05$)	4.71M	–	90.8	87.1
Try 8⋆	SWA: 40 more epochs from Try 4, 35 snapshots averaged	4.71M	–	91.4	86.7

⋆Selected final submissions. [a]98.02 with flip TTA. Try 8 before fixing its BatchNorm statistics: 84.8 public / 84.7 private.

Table 2: All submitted models (accuracy in %). Mean over Tries 2–8: 90.8 public, 87.0 private.

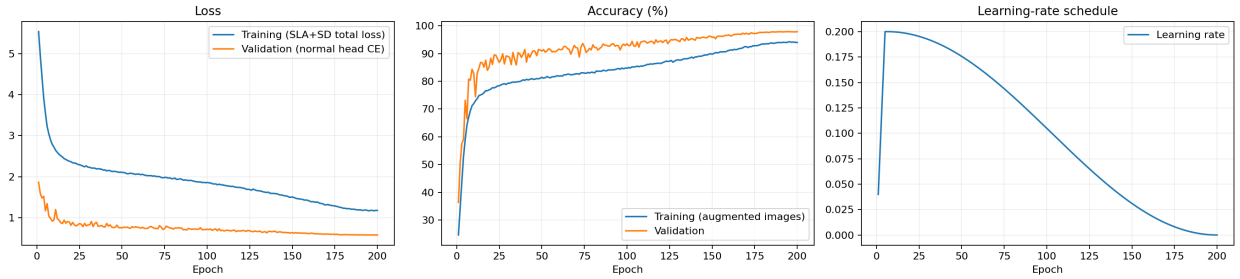


Figure 1: Training curves of Try 2, as saved by its notebook: loss (left), training and validation accuracy (middle) and the learning-rate schedule (right). Most of the validation gain arrives in the last third of training, as the learning rate decays towards zero. Training accuracy is lower than validation accuracy because training images are heavily augmented.

Final result. Our selected submissions were Try 4 and Try 8, the two best on the public leaderboard. Both are single ResNets with **4,707,442 parameters**; their hidden-test accuracies are **91.5% / 91.4% (public)** and **86.5% / 86.7% (private)**, so our final private-leaderboard score is **86.7%**. The best private score among our submissions was Try 3 (87.7%).

4.1 What the follow-up experiments showed

Knowledge distillation. Distillation [10, 11] lets a same-size student learn from the averaged soft predictions of trained teachers (temperature 4, weight 1). With two teachers (Try 4) the public score rose to 91.5%, our best. Adding teachers did *not* help: three teachers gave 91.0% and four gave 89.6%. Comparing prediction files explains part of this: Try 6 shifted about 15 test images from "cat" to "dog", the hardest pair in CIFAR-10. Three of the four teachers had trained on the same 50k images, so on those images they were confident and nearly identical; more such teachers push the soft targets back towards hard labels, removing the information distillation is meant to transfer. Privately, distillation was neutral (Try 3 without it: 87.7%; Try 4 with it: 86.5%).

Sharpness-Aware Minimization (SAM). SAM [12] takes each gradient step from a nearby "worst-case" point to favour flat minima. Added to Try 4's recipe, it gave 90.8% public and 87.1%

private at twice the cost per step: no measurable effect.

Stochastic Weight Averaging (SWA). SWA [13] continued Try 4 for 40 epochs at a constant learning rate of 0.01 and averaged 35 per-epoch weight snapshots into one model. Its first submission scored only 84.8%. The weights were fine; the BatchNorm statistics were not. Standard SWA recomputes them on clean, upright images, but our model is always trained on batches containing all four rotations, so its layers expect very different activation statistics. Recomputing them on augmented, four-way-rotated training batches (no labels, no test data) changed 185 of 2,000 predictions and restored 91.4% public / 86.7% private, a tie with Try 4.

5 Lessons Learned

1. **Most accuracy is earned at the end of the schedule.** Judging a run before its learning rate has decayed, or stopping it early, gives a misleading answer. "Number of epochs" matters far less than reaching the low-learning-rate phase.
2. **Ideas that change *what* the network learns beat ideas that change *how* it is optimized.** Rotation self-supervision and augmentation (+6 points private) changed the features. Distillation, SAM, SWA and EMA, which mostly change the optimization path, were neutral once the recipe was strong. With a fixed, small parameter budget the representation, not the optimizer, was the bottleneck.
3. **Validation accuracy is an optimistic proxy on a shifted test set.** Our 98% validation corresponded to 87% private accuracy. Validation remained useful for comparing variants of the same recipe, but its absolute value said little about test accuracy, and ideas aimed at robustness (augmentation, auxiliary self-supervision) were the ones that transferred.
4. **A 1,000-image leaderboard cannot rank models closer than about 1 point.** Seven runs of essentially one recipe spanned 89.6–91.5% public but only 86.5–87.7% private, and the two rankings were close to unrelated (Table 2: the run with the best public score, Try 4, has the lowest private score). Several of our "improvements" were public-leaderboard noise.
5. **The winner's curse.** When many near-equal models are submitted, the one with the highest public score is usually the one that was luckiest on the public half, and it regresses on the private half. That is exactly what happened to Try 4 (best public, nearly worst private). A sounder selection rule would have been: compare variants on a clean validation split, and choose final submissions that are *different* models (e.g. Try 3 and Try 4) rather than the top two public scores of near-identical ones.
6. **Methods have hidden assumptions.** SWA's standard BatchNorm recalibration silently assumes the model was trained on clean inputs; with rotation self-supervision it cost 7 points until fixed. Every added component should be checked against the rest of the pipeline, ideally on validation, before it reaches the leaderboard.
7. **More of a good thing is not always better.** Two distillation teachers helped publicly; four, all trained on the same data, hurt. Teachers need to disagree to be informative.
8. **Engineering is part of the method.** Fast in-memory data loading, time projections, per-epoch checkpoints and exact resumption turned unreliable free GPUs into a usable experimental platform.

6 Conclusion

Under a 5M-parameter budget, a standard 4/4/3 ResNet trained with a complete cosine schedule, rotation-based self-supervised label augmentation, TrivialAugment, Cutout and label smoothing improved hidden-test accuracy by about 6.5 points over our baseline (81.2% → 87.7% private, 85.3% → 91.5% public). Further optimization-side techniques did not yield measurable gains on unseen data, and selecting final models by public score cost us about one point of private accuracy. With more time we would tune the recipe only on a held-out split, compare candidates with multiple seeds, and spend the remaining budget on architectural or data-side changes that target the train/test distribution shift.

Acknowledgements. The course-suggested repository `kuangliu/pytorch-cifar` [14] was consulted as a reference.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. *CVPR*, 2016.

[2] B. Recht, R. Roelofs, L. Schmidt, V. Shankar. Do ImageNet classifiers generalize to ImageNet? *ICML*, 2019 (includes the CIFAR-10.1 test set).

[3] H. Lee, S. J. Hwang, J. Shin. Self-supervised label augmentation via input transformations. *ICML*, 2020.

[4] S. G. Müller, F. Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. *ICCV*, 2021.

[5] T. DeVries, G. W. Taylor. Improved regularization of convolutional neural networks with Cutout. arXiv:1708.04552, 2017.

[6] C. Szegedy et al. Rethinking the Inception architecture for computer vision (label smoothing). *CVPR*, 2016.

[7] I. Loshchilov, F. Hutter. SGDR: Stochastic gradient descent with warm restarts. *ICLR*, 2017.

[8] T. He et al. Bag of tricks for image classification with convolutional neural networks. *CVPR*, 2019.

[9] A. Tarvainen, H. Valpola. Mean teachers are better role models. *NeurIPS*, 2017.

[10] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[11] T. Furlanello et al. Born-again neural networks. *ICML*, 2018.

[12] P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *ICLR*, 2021.

[13] P. Izmailov et al. Averaging weights leads to wider optima and better generalization. *UAI*, 2018.

[14] K. Liu. pytorch-cifar. https://github.com/kuangliu/pytorch-cifar.